ORGANIZAÇÃO E ARQUITETURA DE COMPUTADORES

ASSEMBLY

Instruções de máquina e o MIPS

Prof. Júlia Roberta Quoos Alves

Mini-aula · 10 min

Faculdade Antonio Meneghetti

O PROCESSADOR ENTENDE C?

Ele só executa sequências de bits. Quem faz a ponte entre o código e o hardware?

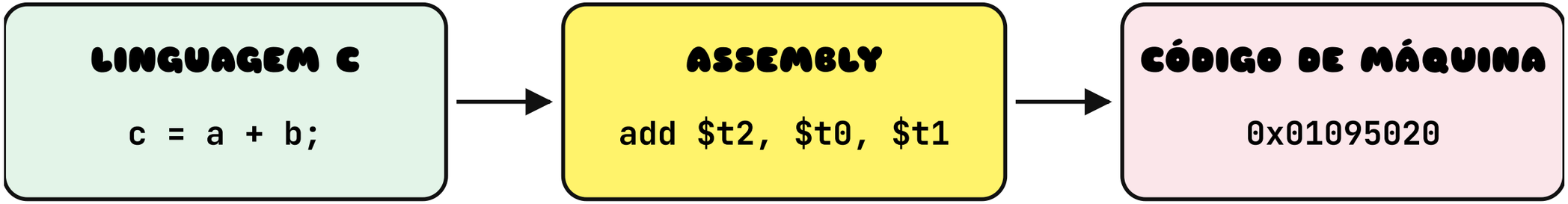
SUMÁRIO

1. **Instruções de máquina**
   o que é assembly · elementos de uma instrução · RISC x CISC · o que é o MIPS
2. **Exemplo de código em MIPS**
   do assembly ao binário · simulador

Objetivo da aula
Relacionar uma instrução MIPS com a sua codificação e ler um trecho simples de código.

1 INSTRUÇÕES DE MÁQUINA

A CPU FAZ O QUE AS INSTRUÇÕES MANDAM



A operação de uma CPU é determinada pelas instruções que ela executa. Cada linha de assembly corresponde a **uma instrução de máquina**.

ELEMENTOS DE UMA INSTRUÇÃO

Código de operação
qual operação fazer (soma, leitura da memória, desvio).

Operandos fonte
de onde vêm os dados de entrada.

Operando de destino
onde guardar o resultado.

Próxima instrução
onde buscar a seguinte. No MIPS fica implícita: PC + 4, exceto nos desvios.

Cada instrução precisa conter toda a informação para a CPU executá-la.

RISC X CISC

RISC

Poucas instruções simples e de **tamanho fixo** (32 bits no MIPS). Memória só por **lw** e **sw**. Pensada para o pipeline e para a otimização pelo compilador. Ex.: MIPS, ARM, RISC-V.

CISC

Muitas instruções, de **tamanho variável** (1 a 15 bytes no x86). Uma instrução pode fazer várias operações e acessar a memória direto. Ex.: x86.

Somar um valor da memória: x86 faz **add eax, [ebx]**; MIPS faz **lw** e depois **add**.

O QUE É O MIPS?

32
registradores
de 32 bits cada ($t0, $s0, $zero...)

32 bits
por instrução
sempre o mesmo tamanho

3
formatos
R (registradores), I (imediato), J (salto)

Arquitetura de conjunto de instruções (ISA) do tipo **RISC**, criada em Stanford nos anos 1980. Base do livro de Patterson e Hennessy e parente direto do RISC-V.

2 EXEMPLO DE CÓDIGO EM MIPS

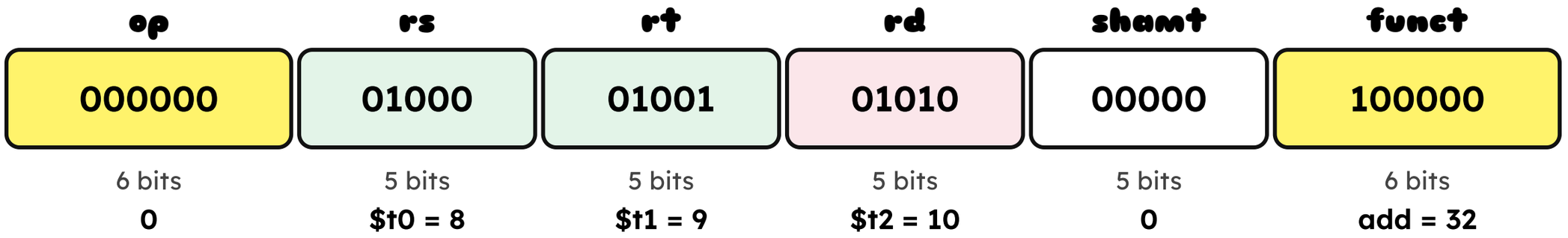
SOMANDO 5 + 7

```
li   $t0, 5            # $t0 = 5
li   $t1, 7            # $t1 = 7
add  $t2, $t0, $t1     # $t2 = 12
```

li é uma pseudoinstrução: o montador troca por **addiu $t0, $zero, 5**.

add é formato R: três registradores.

DO ASSEMBLY AO BINÁRIO



add $t2, $t0, $t1 = 000000 01000 01001 01010 00000 100000 = **0x01095020**.

O op 0 diz “formato R”; o funct diz qual operação.

NO SIMULADOR

ohtopico.com.br/simuladores/mips.html

- montar o programa e ver o li virar instrução real;
- clicar na linha do **add** e conferir os campos e o binário;
- executar passo a passo e ver **$t2 = 12** nos registradores.

Mini-MIPS: simulador aberto e gratuito, com formatos R, I e J.

CONFIRA

1. Em **add $t2, $t0, $t1**, quais campos dizem à CPU que a operação é uma soma?
2. Trocando por **sub $t2, $t0, $t1**, o que muda no binário?
3. Por que **li** não aparece no código de máquina?

RESUMO DA AULA

- Uma instrução de máquina é uma **sequência de bits** que carrega a operação e os operandos.
- Assembly é a **forma legível** dessas instruções; cada arquitetura, RISC ou CISC, tem o seu.
- MIPS é uma **arquitetura de conjunto de instruções RISC**: 32 bits, formatos R, I e J.

REFERÊNCIAS

- PATTERSON, D. A.; HENNESSY, J. L. **Organização e projeto de computadores**: a interface hardware/software. 4. ed. Rio de Janeiro: Elsevier, 2014.
- HENNESSY, J. L.; PATTERSON, D. A. **Arquitetura de computadores**: uma abordagem quantitativa. 5. ed. Rio de Janeiro: Elsevier, 2014.
- STALLINGS, W. **Arquitetura e organização de computadores**. 10. ed. São Paulo: Pearson, 2017.
- CARTER, N. **Arquitetura de computadores**. Porto Alegre: Bookman, 2003. (Coleção Schaum).

OBRIGADA

Júlia Roberta Quoos Alves

Material e simuladores abertos em **ohtopico.com.br**